

* Reasons for Studying Programming languages

1. Increased ability to express ideas:

- It is widely believed that ~~the~~ programmer thinking capability is influenced by the expressive power of the language.
- It is difficult for people to conceptualize structures, they can't describe verbally or in writing.
- If a programmer aware of many languages feature, then we can reduce limitations.

2. Improved background for choosing appropriate languages:

- If a programmer know only one language, then he will ^{use} that language even it is not suitable.
- If a programmer know several languages, then he can choose best one.

3. Provides greater ability to learn new languages:

- knowing fundamental concepts makes learning new languages easier.
- If a programmer know object oriented programming, then it is easy to understand Java.
- Ph^s are still in a state of continuous evolution
↙ means continuous learning is essential.

Programming languages

4. Understand Significance of implementations

- Understanding of implementation issues leads to an understanding of why languages are designed the way they are. (or) By learning & understanding different languages, Programmer can know the reasons behind the design.
- ~~that~~ It leads to use a language more intelligently.

5. Overall advancement of computing

- Many believe that ALGOL 60 was a better language than Fortran.
- However Fortran was most widely used.
- ~~or~~ ALGOL 60 is difficult for understanding to the programmers.

6. Ability to design new languages

- The more languages you gain knowledge of the better understanding of programming languages concepts you understand.

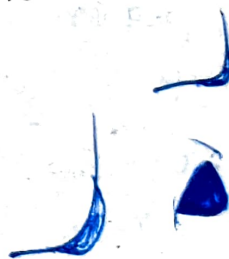
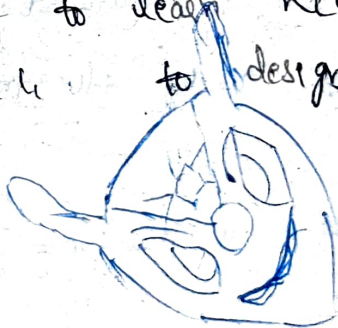
→ Increased ability to express ideas

→ improved background for choosing a

→ Un — — — — —

→ ability to learn new h

→ ability to design h.



* Programming Domain

- The appropriate / suitable language to use often depends on the application domain ~~create the problem~~
- So different languages are preferred for taking care of various computer applications.
- It is very important for developers to pick up the best or suitable language,
- It defines the ability of using a specific language for a specific language.
- Some programming domains are as follows:

1. Scientific Applications.
2. Business Applications
3. Artificial Intelligence
4. System Programming
5. Web Software.

1. Scientific Applications

- Scientific applications require large no. of floating point computations.
- It typically makes use of arrays.
- The first language used for scientific application was FORTAN.
- The syntax of FORTRAN is similar to mathematics.
- Scientists find it easy to use.

2. Business Applications

- COBOL was the first language used for Business applications.
- For building the business application COBOL is used.
- It requires use decimal numbers & characters data.
- Java is also a great choice for business applications.

3. Artificial Intelligence:

- AI is a huge topic
- It is a broad area of computer applications.
- AI is an area in which the computations are symbolic.
- Symbolic computation means that symbols, consisting of names rather than numbers,
- LISP is the first language developed for AI applications,
- And then PROLOG was invented.

4. Systems Programming

- It is a system software which consists of collection of OS & programming support tools.
- The system s/w is used continuously, hence it needs to be efficient.
- System s/w must have low-level features.
- C, C++ are usually preferred for system programming.
- C is the most widely used language.

5. Web Software

- It needs a support for variety of languages from markup languages, scripting language, general-purpose language.
- HTML, PHP, Java are used.

4. Understand Significance

* Language Evaluation Criteria

- A programming language possess many characteristics.
- These characteristics can be evaluated based on the language Evaluation Criteria.
- Language Evaluation Criteria on programming languages are set of principles and standards.
- It is used to assess/determine the quality and effectiveness of programming languages.
- Steps involved in the language evaluation criteria are:
 1. Readability
 2. Writability
 3. Reliability
 4. Cost.

1. Readability

- It refers to how easy it is to read and understand code written in a particular language.
- Important criteria is "ease of use".
- A language with good readability, has a syntax that is easy to comprehend.
- A language with poor readability can lead to confusion & errors in code.

Characteristics of Readability.

- The following characteristics are also included in readability:
 - Simplicity and orthogonality.
 - Control Statements
 - Datatypes and structures.

②. Simplicity and orthogonality

- It strongly affects readability.
- Too many features make the language difficult to learn.

Ex: `Count = Count + 1;`

or

`Count++`

`++Count`

③. Orthogonality

- It makes the language easy to learn & read.
- The lack of orthogonality leads to exceptions to the rules of the language.
- The more orthogonal the design of language, the fewer exceptions the language rules require.

④. Control Statements

- Program that can be read from top to bottom is easier than a program that requires jump from one statement to some statement.
- Hence the use of GOTO statement must be ~~restricted~~ ^{readable} to make a program more ~~read~~.

⑤. Data types and Structures

- Language's readability is also affected by the data types and data structures.
- A language which doesn't support boolean data type uses numeric type.
- For a language which ^{doesn't} support 'bool' data type,
`timeout = 1`
- For a language that ~~doesn't~~ support 'bool': `timeout = true`.

2. Writability

- A language with good writability has a concise syntax.
- Powerful abstractions are also important for writability.
- It is a measure of how easily a language can be used to create programs for a chosen problem domain.
- Characteristics that affect readability also affect writability.
- The following characteristics are included in writability:
 - Simplicity
 - Orthogonality
 - Support for Abstraction
 - Expressivity.

3. Reliability

- A Program is said to be reliable, if it performs to its specifications under all conditions.
- It is the ability of a language to produce consistent & predictable results even in the presence of errors.
- A language with good reliability will produce less unexpected results or crashes.
- Characteristics that affect reliability:
 - Type checking: checking for type errors which occurs during compilation & program executions.
 - Exception handling: handling run-time errors.
 - Readability
 - Maintainability.
 - Aliasing: Single memory is shared by 2 or more names.

4. Cost:

→ The total cost of a programming language can be determined by following factors:

- The cost of training programmers to use language.
- The cost of writing programs.
- The cost of compiling programs.
- The cost of executing programs.
- The cost of maintaining programs.
- The cost of implementing programs.
- The cost of poor reliability.

Total Cost = Cost of training programmers + Cost of writing programs + ...

* Influences on Language Design

→ The factors that influence on language design are as follows:

1. Computer architecture
2. Programming methodologies.

1. Computer architecture

→ Von Neumann is the most commonly used

Computer architecture.

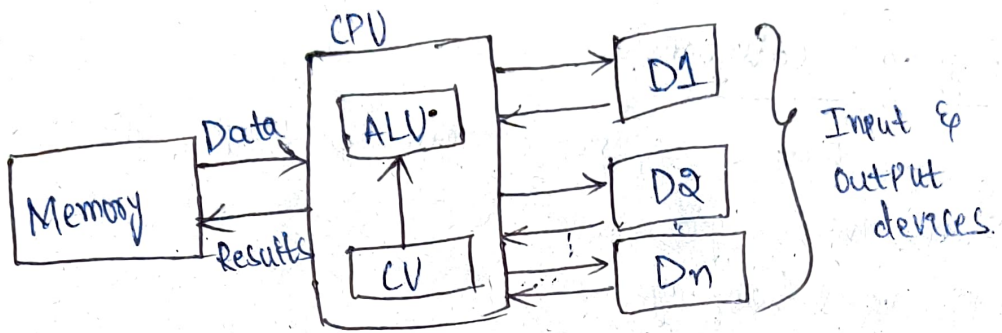
→ we use imperative language.

→ According to this architecture data and programs are stored in memory.

→ Memory is separate from CPU.

→ Instructions and data move from memory to CPU.

→ Results of operations on the CPU move back to memory.

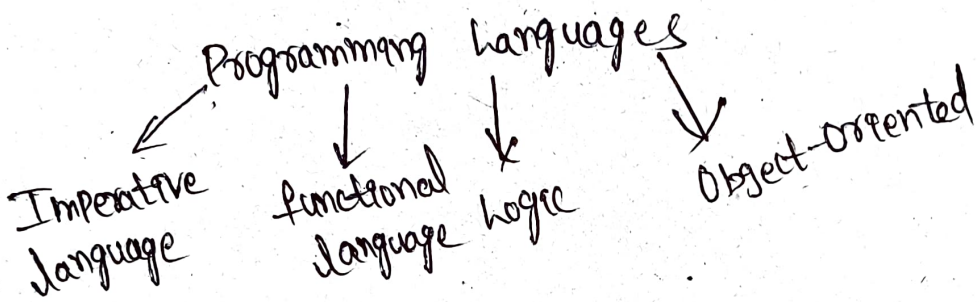


Non-Neumann Computer Architecture.

* Language Categories

→ Languages are categorized into different types:

1. Imperative languages
2. Functional language
3. Logic programming language
4. Object Oriented programming.
5. Special-Purpose Languages.



1. Imperative Language

- It is also called as Procedural language.
- A program consists of sequence of statements.
- After execution of each statement the values are stored in the memory.
- The central features of this language are variables, assignment statements & iterations.
- Example of imperative languages are:
C, Pascal, Ada, Fortran, Cobol, Algol etc..

Syntax:

Statement 1;

Statement 2;

2. Functional Language

- It is also called as Applicative language.
- the computations are ~~expressed~~ done by applying functions.
- Programming in functional languages can be done without using any kind of variables, assignment statements & iterations.
- Computations of functional languages are performed largely through applying functions to values.

Ex: $(+ 10 20)$

$$(10 + 20) = 30$$

Syntax $\text{function}_n(\dots \text{function}_2(\text{function}_1(\text{data})) \dots)$

- Examples of functional programming are: LISP, ML, Haskell,

3. Logic Programming Language

- Also known as Rule-based language.
- No specific order is ~~following~~ followed.
- PROLOG is the most common rule-based language.
- Execution is similar to imperative language except that the statements are not sequential.
- Enabling conditions determine the order of execution.

Syntax:

enabling condition 1	→	Action 1
enabling condition 2	→	Action 2
⋮		⋮
enabling condition n	→	Action n

Ex: PROLOG

4. Object oriented programming language

- In OOPS, everything is considered as objects.
- Some core concepts of OOPS include:
 - Class
 - Abstraction
 - Objects
 - Encapsulation
 - Inheritance
 - Polymorphism

Ex: Java, Ada, C++

* Language Design Trade-offs

- Some of the language design trade-offs are as follows:

1. Readability Vs Cost of Execution
2. Maintainability Vs Readability
3. Flexibility Vs Safety

* Programming Environments

- Programming environment is an environment in which the programs can be created & tested.
- It refers to the collection of tools used for developing S/W.
- The collections include file system, linker, text editor & compiler.
- The Programming Environment influence on language design in :- Separate compilation
 - Testing
 - Debugging.

→ There are several programming environments:

1. UNIX

2. Borland JBuilder

3. Microsoft Visual Studio

4. NetBeans.

1. UNIX

→ It is an older programming environment.

→ It is portable multiprogramming OS.

2. Borland JBuilder

→ It is a programming environment.

→ used for the development of Java.

→ It provides debugger, editor, ~~and~~ compiler & file system.

3. Microsoft Visual Studio

→ It is a recent programming environment.

→ It is a recent innovation in software development environments.

→ It consists of huge collection of software tools.

4. NetBeans

→ It mainly used for Java application development.

→ It is one of the programming environments.

→ It also support JavaScript, Ruby & PHP.

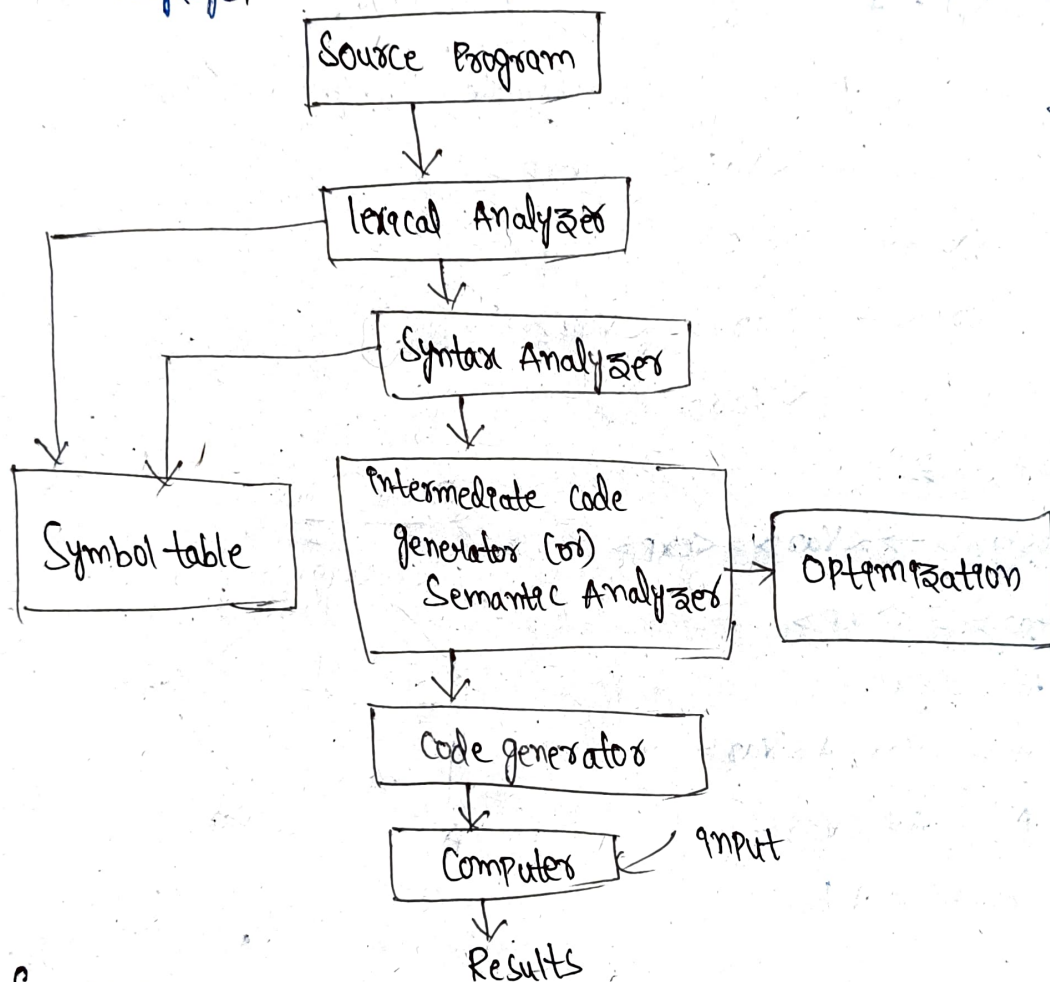
* Implementation methods

→ There are three implementation methods available:

1. Compilation
2. Pure interpretation
3. Hybrid.

1. Compilation

→ Firstly, the source code is converted into machine code language.



Source Programs:

→ The program written by users are called Source Programs.

Lexical Analyzer:

→ It will take source code & convert into lexical units.

Syntax Analyzer:

→ It will construct trees called Parse trees with the help of lexical units.

Semantic Analyzer:

→ It will check for errors and bugs.

Optimization:

→ If a program is complex, then it is converted into

simple one.

Symbol table:

→ It serves as Database for compilation.

Code generators:

→ It will convert into equivalent machine language.

2. Pure Interpretation

→ A program is interpreted by a collection of programs called "Interpreter."

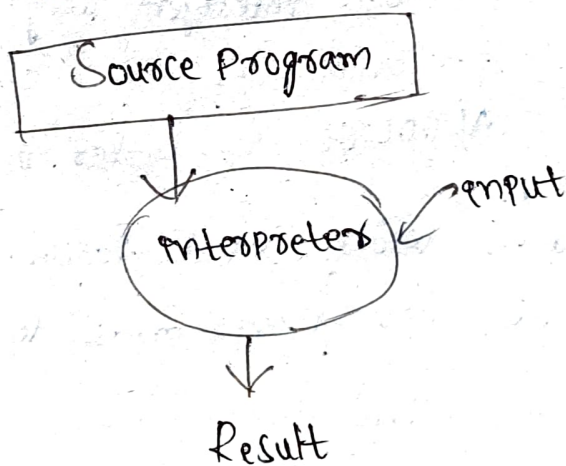
→ There is no need of intermediate code generator.

→ Interpreter will compile the code line by line in a sequence manner.

Disadvantages

→ It requires more space.

→ execution is slower 10 to 100 times than compiler



3. Hybrid Implementation System

- Some language have the implementation system that are ~~are~~ actually the combination of compilers and interpreter.
- Compiler translate high-level language Programs to an intermediate language designed to allow easy interpretation.
- The interpreter, interprets the intermediate language ~~from~~ form the Machine Code.

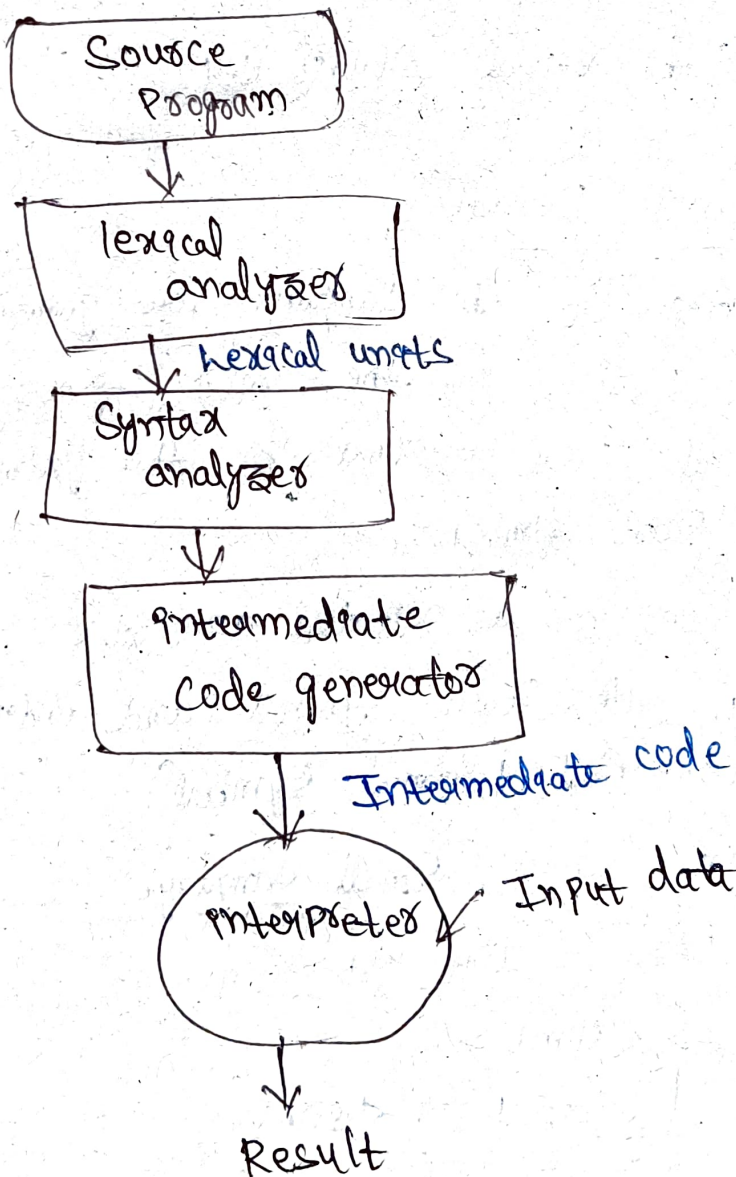


Fig: Hybrid implementation System.

Part II : Syntax and Semantics

1.10 General Problem Describing Syntax

Q.29 Explain the following terminologies –

i) Sentence, ii) Language, iii) Lexeme, iv) Token.

Ans. : i) Sentence : Sentence is a string of characters over some alphabet.

ii) Language : Language is a set of sentences.

iii) **Lexeme** : Lexeme is a lowest level syntactic unit of language. For example – index, count

iv) **Token** : The token is a category of lexemes. For example – identifier, keywords.

Consider following statement

$a = b + 10$

Lexeme	Token
a	identifier
=	assign_operator
b	identifier
+	addition_operator
10	int_literal

Q.30 Define syntax and Semantics.

 [JNTU : Part A, Nov.-15, Dec.-15, Marks 2]

Ans. : Syntax : It is the form or structure of the expressions, statements, and program units. The grammar of a language is called syntax.

Semantics : The meaning of the expressions, statements, and program units is called semantic.

Q.31 Differentiate between syntax and semantics.

 [JNTU : Part B, March-16, Marks 5]

Ans. :

Syntax	Semantics
The form or structure of the expressions, statements, and program units is called syntax .	The meaning of the expression, statements and programming units is called semantics .
It is handled at compile time .	It is handled at run time .
It is given by context free grammar	It is given by syntax directed definition .
The syntactic errors are easy to locate .	The semantic errors are difficult to locate .

Q.32 Define grammars, derivation and

Ans. : Grammar : The formal language-generation mechanism that are used to describe the syntax of programming language is called grammar. There are two methods of describing the syntax - Backus-Naur Form(BNF) and Context Free Grammar(CFG).

Derivation : Refer Q.34.

Parse Tree : Refer Q.34.

Q.33 Discuss about language recognizers and language generators.

 [JNTU : Part B, Dec.-16, Marks 5]

Ans. : For defining a language formally there are two ways - i) Recognition and ii) Generation.

i) Language Recognizers :

- A recognition device reads input strings of the language and decides whether the input strings belong to the language.
- For example - consider an input set $\Sigma = \{a,b\}$ and language L consists of only even length strings, the language recognizer will indicate whether the string 'abbb' belongs to L and 'abb' does not belong to L.

ii) Language Generators :

- A language generator is a device that can be used to generate the sentences of a language.
- One can determine if the syntax of a particular sentence is correct by comparing it to the structure of the generator.

Q.34 Define Derivation and Parse Tree.

 [Part A, Nov.-15, Marks 3]

Ans. : Derivation : Derivation is a repeated application of rules, starting with the start symbol and ending with a sentence (all terminal symbols).

A leftmost derivation is one in which the leftmost nonterminal in each sentential form is the one that is expanded. The derivation continues until the sentential form contains no non terminals.

A rightmost derivation is one in which the rightmost nonterminal in each sentential form is the one that is expanded. The derivation continues until the sentential form contains no non terminals.

For example : Consider following grammar -

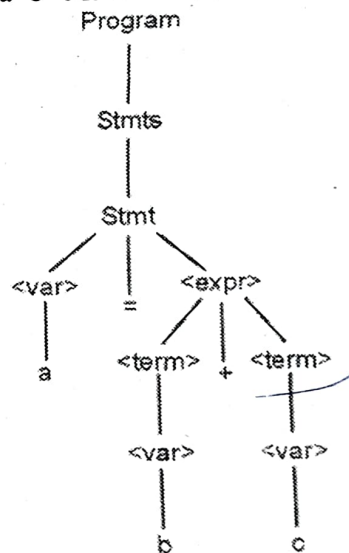
```
<program> → <stmts>
<stmts> → <stmt> | <stmt> ; <stmts>
<stmt> → <var> = <expr>
<var> → a | b | c | d
<expr> → <term> + <term> | <term> - <term>
<term> → <var> | const
```

The derivation of a statement $a=b+c$ is as follows -

```
<program> => <stmts> => <stmt>
=> <var> = <expr>
=> a = <expr>
=> a = <term> + <term>
=> a = <var> + <term>
=> a = <var> + <var>
=> a = <b> + <term>
=> a = <b> + <c>
```

Parse Tree : Hierarchical structures of the language are called parse trees. For example -

Parse tree for $a=b+c$ is as follows -



Q.35 Explain the parse tree for the sum and average program by using grammar.

 [JNTU : Part B, March-16, Marks 5]

Ans. : Step 1 : We will write the grammar rules before building the parse tree for the sum and average program.

```
program → stmt_list $$
stmt_list → stmt stmt_list | ε
stmt → id := expr | read id write | expr
expr → term term_tail
term_tail → add_op term term_tail | ε
```


Q.39 Explain the Backus Naur Form. [JNTU : Part B, Marks 5]

Ans. : • Backus Naur form is a representation of context free grammar in which particular notations are used.

The non terminals in BNF are enclosed within special symbols < and >

- The empty string is written as <empty>
- The terminals appear as it is. Sometimes they can be denoted with quotes.
- The **productions** can be denoted using the symbol ::= which means "can be". The symbol "|" means OR can be used to denote the alternative definitions at the right hand side of the production.

- **Example -**

Consider following BNF rules

<stmt> ::= <type> <list>;

<type> ::= int | float

<list> ::= <list>, id

<list> ::= id

Here *stmt*, *type* and *list* are **nonterminal** symbols. The *int*, *float*, *id* and ; are **terminal** symbols. The <stmt> is a **starting nonterminal**.

The alternatives are separated by |

Q.40 What are three reasons why syntax analyzers are based on grammars ?

 [JNTU : Part A, Marks 3]

Fo
Co
E:
Th
tre

Q.42

Ans.
ambig
for thi

Q.43 Describe the lists using BNF notation.

Ans. : Syntactic lists are described using recursion

$\langle \text{id_list} \rangle \rightarrow \text{identifier}$

$|\text{identifier}, > \langle \text{id_list} \rangle$

The above rule is recursive because the LHS symbol appears at the RHS.

Q.44 What is EBNF ? Explain in detail.

 [JNTU : Part B, Marks 5]

Ans. :

- The EBNF stands for Extended BNF.
- The extensions do not enhance the descriptive power of BNF; they only increase its readability and writability.
- There are three extensions are commonly used for denoting the EBNF.
 - The optional parts are placed in brackets []

For example -

BNF	EBNF
$\langle \text{if_stmt} \rangle \rightarrow \text{if } \langle \text{expr} \rangle \langle \text{statement} \rangle$ $ \text{if } \langle \text{expr} \rangle \langle \text{statement} \rangle$ $\text{else } \langle \text{statement} \rangle$	$\langle \text{if_stmt} \rangle \rightarrow \text{if } \langle \text{expr} \rangle \langle \text{statement} \rangle [\text{else } \langle \text{statement} \rangle]$

- Put the repetitions in in braces ({})

For example -

BNF	EBNF
$\langle \text{id_list} \rangle \rightarrow \text{identifier}$ $ \text{identifier}, > \langle \text{id_list} \rangle$	$\langle \text{id_list} \rangle \rightarrow \langle \text{identifier} \rangle \{ , \langle \text{identifier} \rangle \}$

- Put multiple-choice options of RHSs in parentheses and separate them with vertical bars

For example -

BNF	EBNF
$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle$ $ \langle \text{term} \rangle / \langle \text{factor} \rangle$ $ \langle \text{term} \rangle \% \langle \text{factor} \rangle$	$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle (* / \%) \langle \text{factor} \rangle$

Ambiguity

- A context free grammar is known as ambiguous if there exists two or more distinct parse trees.
- Ambiguity for simple assignment statement

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{exp} \rangle$

$\langle \text{id} \rangle = A | B | C$

$\langle \text{exp} \rangle \rightarrow \langle \text{exp} \rangle * \langle \text{exp} \rangle$

$\quad | \langle \text{exp} \rangle + \langle \text{exp} \rangle$

$\quad | (\langle \text{exp} \rangle)$

$\quad | \langle \text{id} \rangle$

Ex: $A = B + C * A$

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{exp} \rangle$

$\langle \text{id} \rangle = \langle \text{exp} \rangle$

$A = \langle \text{exp} \rangle$

$A = \langle \text{exp} \rangle * \langle \text{exp} \rangle$

$A = \langle \text{exp} \rangle * \langle \text{id} \rangle$

$A = \langle \text{exp} \rangle * A$

$A = \langle \text{exp} \rangle + \langle \text{exp} \rangle * A$

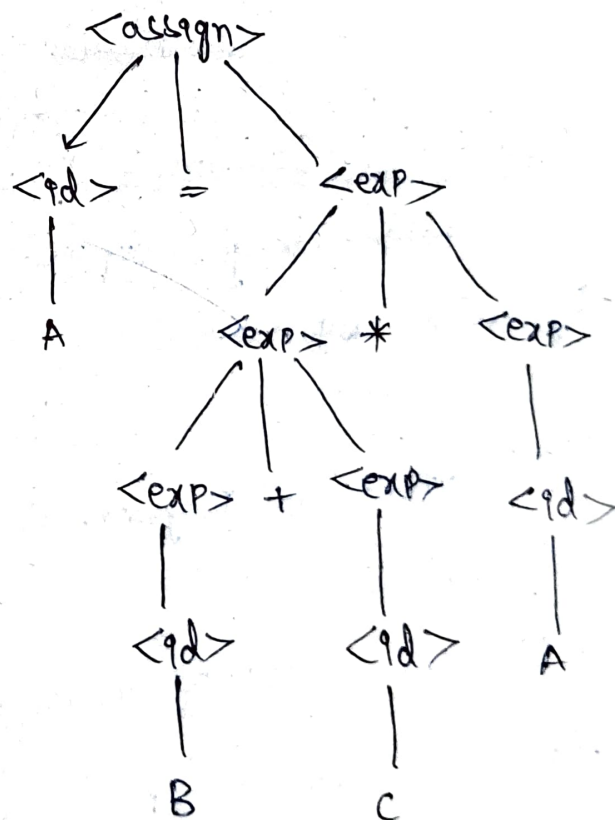
$A = \langle \text{id} \rangle + \langle \text{exp} \rangle * A$

$A = B + \langle \text{exp} \rangle * A$

$A = B + \langle \text{id} \rangle * A$

$A = B + C * A$

Parse tree



$$A = B + C * A$$

Parse tree

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{exp} \rangle$

$\langle \text{id} \rangle = \langle \text{exp} \rangle$

$A = \langle \text{exp} \rangle + \langle \text{exp} \rangle$

$A = \langle \text{id} \rangle + \langle \text{exp} \rangle$

$A = B + \langle \text{exp} \rangle$

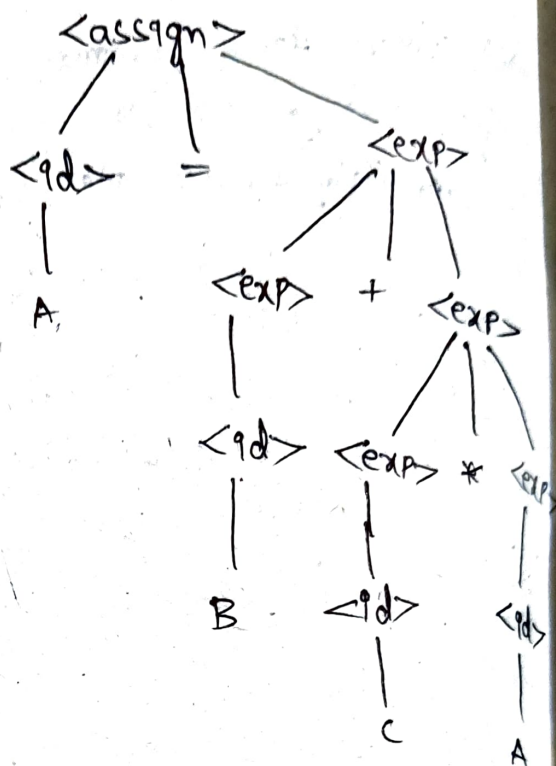
$A = B + \langle \text{exp} \rangle * \langle \text{exp} \rangle$

$A = B + \langle \text{id} \rangle * \langle \text{exp} \rangle$

$A = B + C * \langle \text{exp} \rangle$

$A = B + C * \langle \text{id} \rangle$

$A = B + C * A$



*Differences b/w Operational and Denotational Semantics

Operational Semantics

1. the state changes defined by coded algorithms.

2. It depends on programming languages.

3. It uses state of machine.

4. It is a step by step process.

Denotational Semantics

1. the state changes defined by rigorous mathematical functions.

2. It requires mathematical foundation.

3. It uses state of program.

4. It is not a step by step process.

* Attribute Grammar

Rules

$$A(x) \rightarrow S(x) + I(x)$$

$S(x)$: Synthesized attribute.

$I(x)$: Inherited attribute

$$S(x_0) = f(A(x_1) \dots A(x_n))$$

$$I(x_1) = f(A(x_0) \dots A(x_n))$$

→ Attribute grammar for simple assignment statement.

$$\langle \text{assign} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{exp} \rangle$$

$$\langle \text{var} \rangle \rightarrow A | B | C$$

$$\langle \text{exp} \rangle \rightarrow \langle \text{var} \rangle + \langle \text{var} \rangle$$
$$| \langle \text{var} \rangle$$

Ex: $A = A + B$

$$\langle \text{assign} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{exp} \rangle$$

$$\langle \text{var} \rangle = \langle \text{exp} \rangle$$

$$A = \langle \text{exp} \rangle$$

$$A = \langle \text{var} \rangle + \langle \text{var} \rangle$$

$$A = A + \langle \text{var} \rangle$$

$$A = A + B$$

Parse tree

